

第四章 机器学习

本课程目标



理解机器学习的基本概念



理解文本分类的基本概念



了解机器学习的过程

课程目录

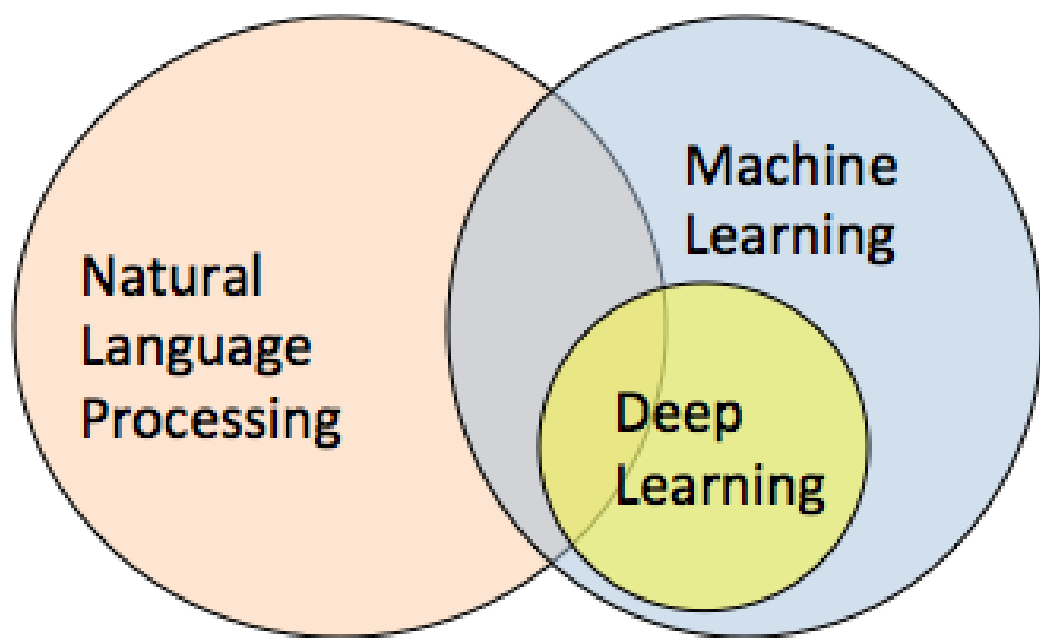
Course catalogue

1/ 机器学习概述

2/ 文本分类概述

3/ 实例

1. 简介



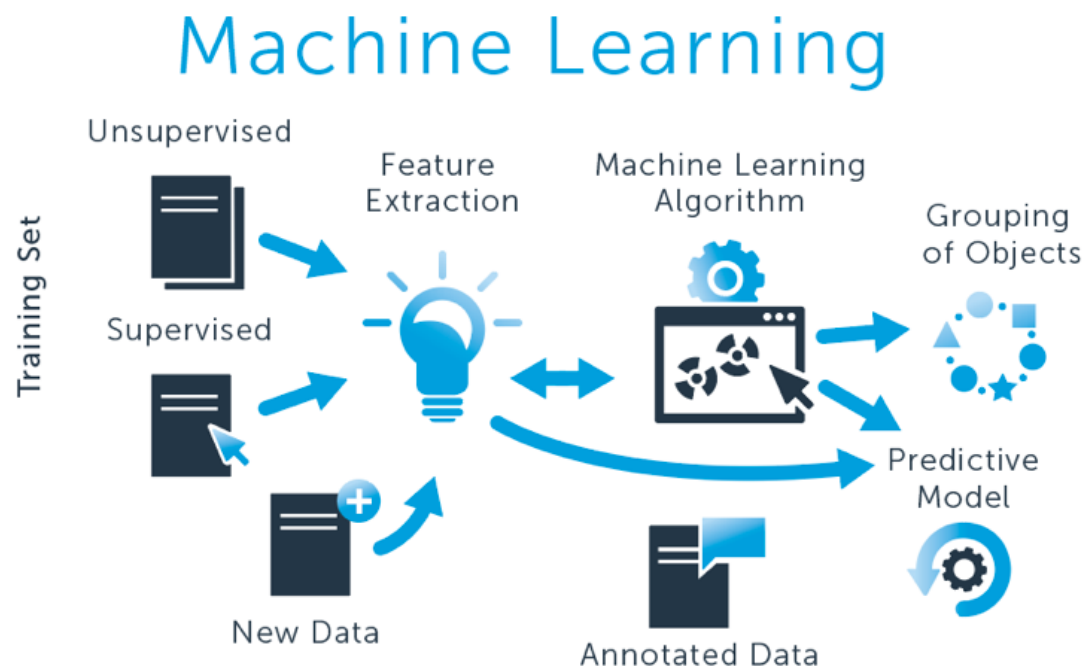
1. 简介

机器学习是一种让计算机利用数据而不是指令来进行各种工作的方法。

“统计”思想将在学习“机器学习”相关理念时无时无刻不伴随，相关而不是因果的概念将是支撑机器学习能够工作的核心概念。

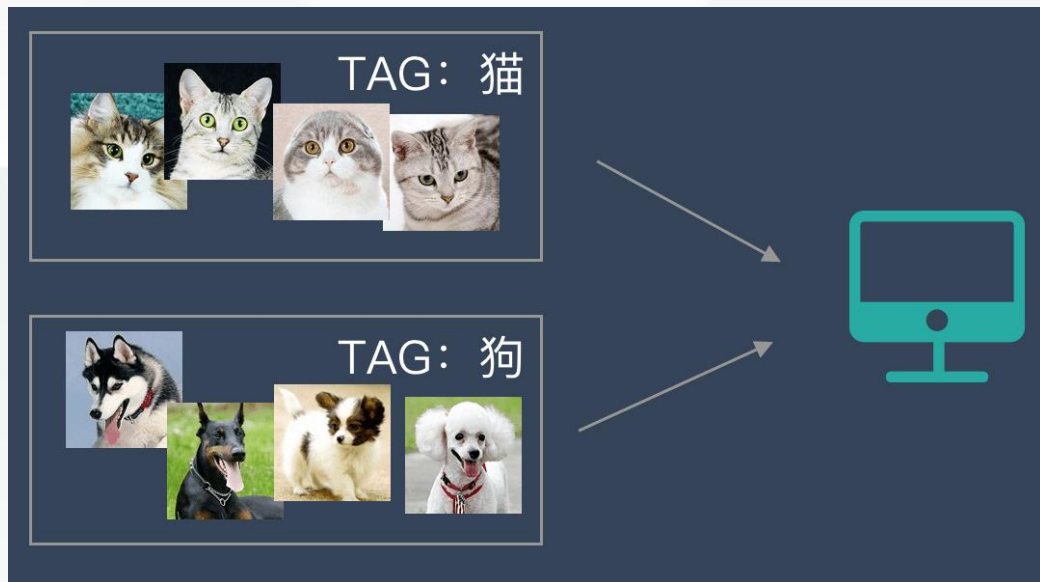
机器学习的定义

机器学习的定义



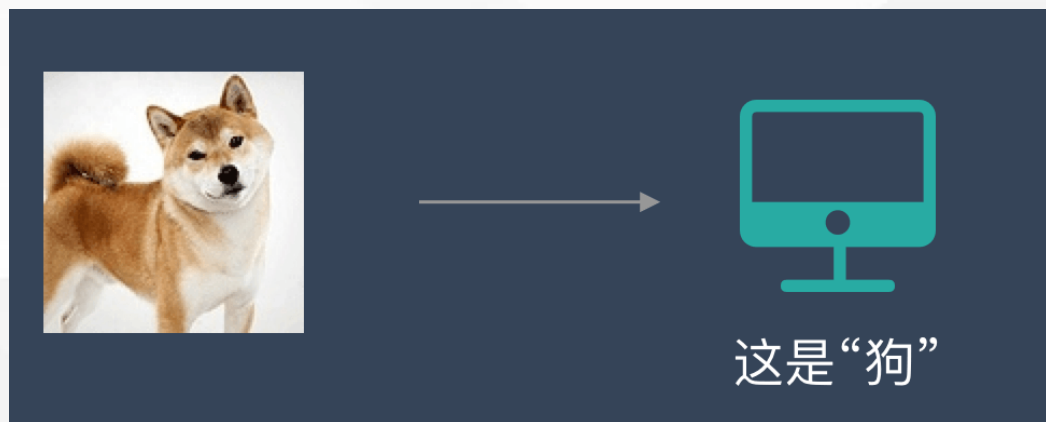
从广义上来说，机器学习是一种能够赋予机器学习的能力以此让它完成直接编程无法完成的功能的方法。但从实践的意义上来说，机器学习是一种通过利用数据，训练出模型，然后使用模型预测的一种方法。

机器学习分类

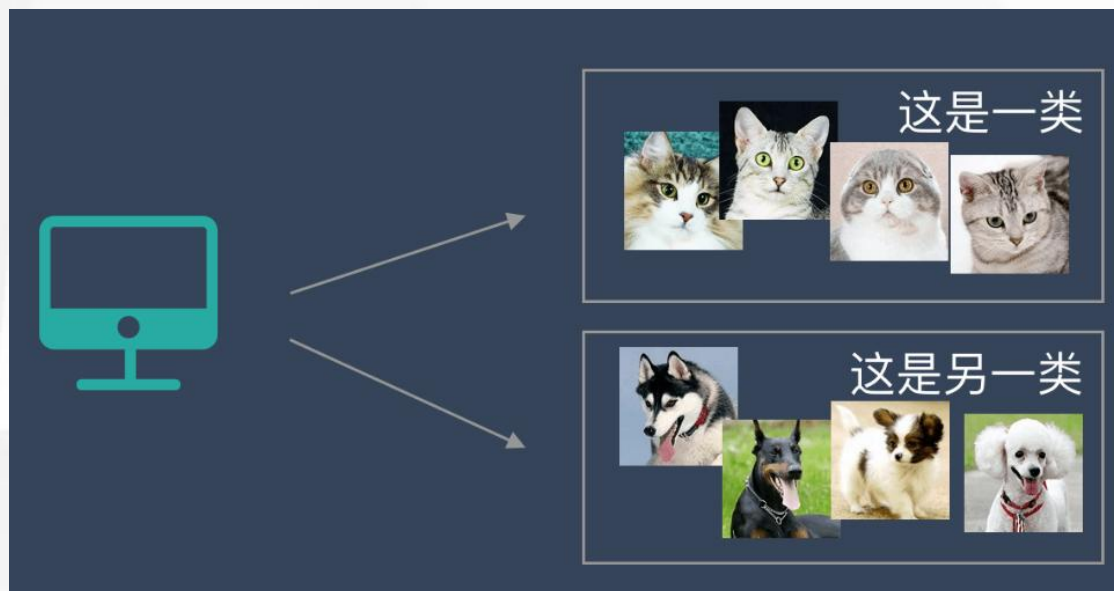
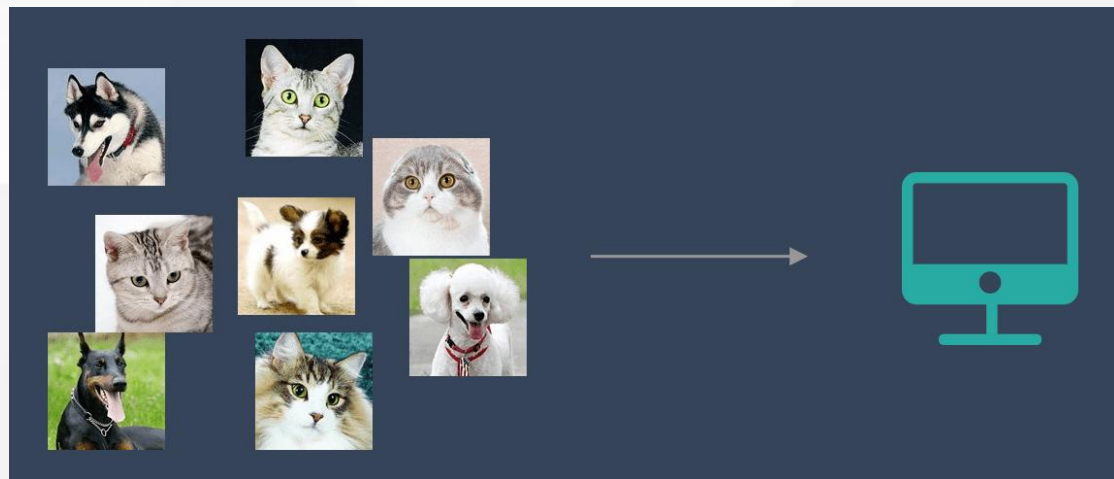


监督学习

监督学习是指我们给算法一个数据集，并且给定正确答案。机器通过数据来学习正确答案的计算方法。



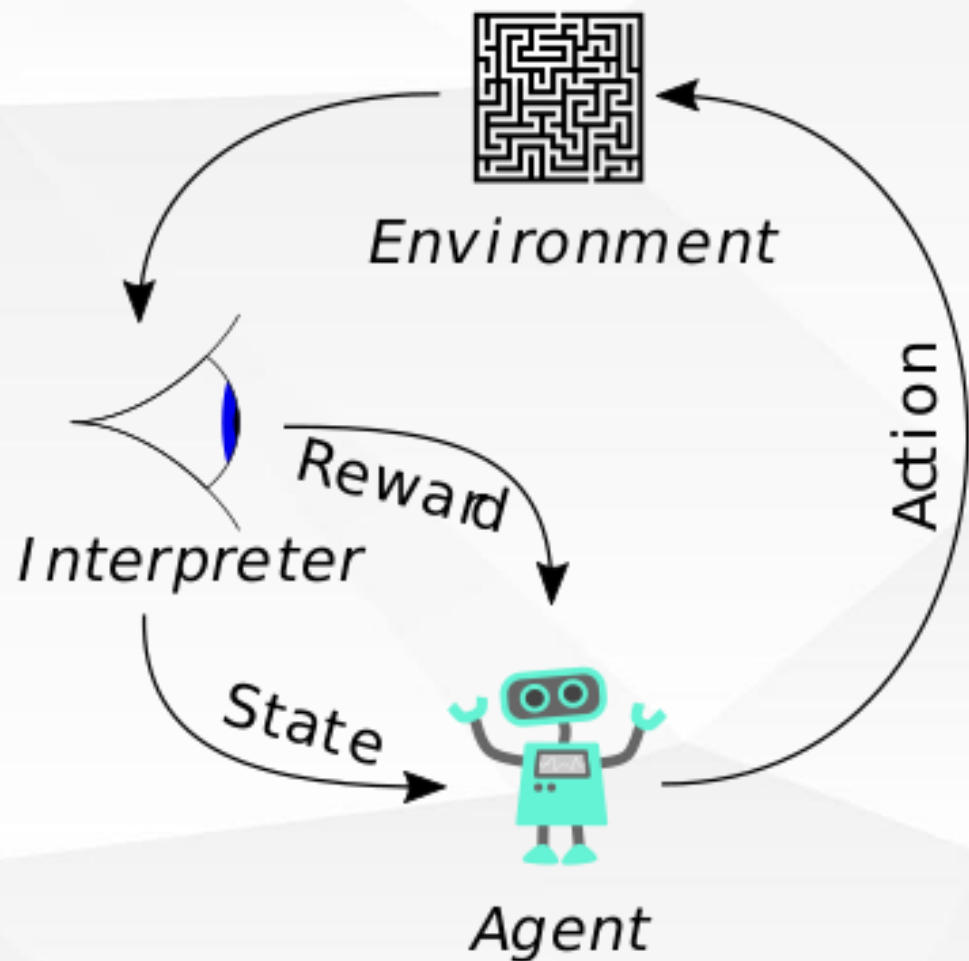
机器学习分类



非监督学习

非监督学习中，给定的数据集没有“正确答案”，所有的数据都是一样的。无监督学习的任务是从给定的数据集中，挖掘出潜在的结构。

机器学习分类



强化学习

强化学习更接近生物学习的本质，因此有望获得更高的智能。它关注的是智能体如何在环境中采取一系列行为，从而获得最大的累积回报。通过强化学习，一个智能体应该知道在什么状态下应该采取什么行为。

监督学习训练

Model

- A parameterized function to map inputs to label
- Model parameters VS hyper parameters
- E.g. listing house → sale price



Loss

- The measure of how good the model does in terms of predicting the outcome
- E.g. classification / regression / contrastive / triplet / ranking

Objective

- E.g. $(\text{predict_price} - \text{sale_price})^2$ **Quadratic Loss Function**

Optimization

- The goal to optimize model params for
- E.g. minimize the sum of losses over examples
- The algorithm for solving the objective

监督学习的类型

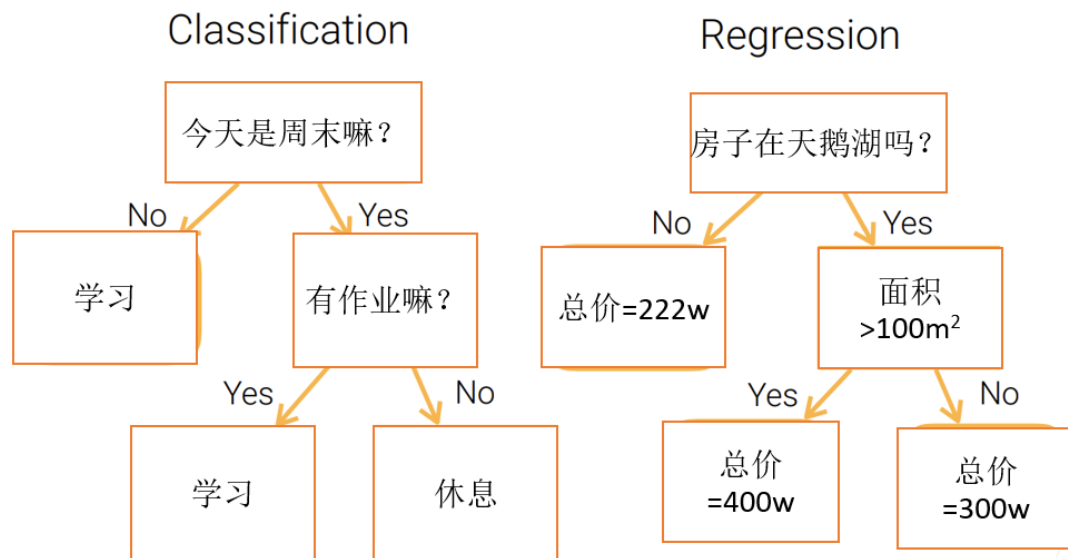
决策树

线性方法

核方法

神经网络

- Use trees to make decisions
- Decision is made from a linear combination of input features
- Use kernel functions to compute feature similarities
- Use neural networks to learn feature representations



优点:

可解释的;
处理数值类和类别类的特征;

缺点:

不稳定;
复杂树, 导致过拟合;
不易于并行化;

机器学习实操的步骤

机器学习实操的步骤

机器学习在实际操作层面一共分为7步：收集数据、数据准备、选择一个模型、训练、评估、参数调整、预测（开始使用）。

机器学习实操的7个步骤



Step 1
收集数据



Step 2
数据准备



Step 3
选择模型



Step 4
训练



Step 5
评估



Step 6
参数调整



Step 7
预估

机器学习实操的步骤

(1) 收集数据

数据是机器学习发展的核心，因为复杂的非线性模型比其他线性模型需要更多的数据。这一步非常重要，因为数据的数量和质量直接决定了预测模型的好坏。

(2) 数据准备

在实际情况中，我们收集到的数据会有很多问题，所以会涉及到数据清洗等工作。

当数据本身没有什么问题后，我们将数据分成3个部分：训练集、验证集、测试集，用于后面的验证和评估工作。

(3) 选择一个模型

研究人员和数据科学家多年来创造了许多模型。有些非常适合图像数据，有些非常适合于序列（如文本或音乐），有些用于数字数据，有些用于基于文本的数据。

机器学习实操的步骤

(4) 训练

数据数量和质量、还有模型的选择比训练本身重要更多。这个过程不需要人来参与，机器独立就可以完成，整个过程就好像是在做算术题。因为机器学习的本质就是将问题转化为数学问题，然后解答数学题的过程。

(5) 评估

一旦训练完成，就可以评估模型是否有用。这是我们之前预留的验证集和测试集发挥作用的地方。评估的指标主要有 准确率、召回率、F值。

(6) 参数调整

完成评估后，您可能希望了解是否可以以任何方式进一步改进训练。我们可以通过调整参数来做到这一点。当我们进行训练时，我们隐含地假设了一些参数，我们可以通过认为的调整这些参数让模型表现的更出色。

(7) 预测

上面的6个步骤都是为了这一步来服务的。这也是机器学习的价值。

课程目录

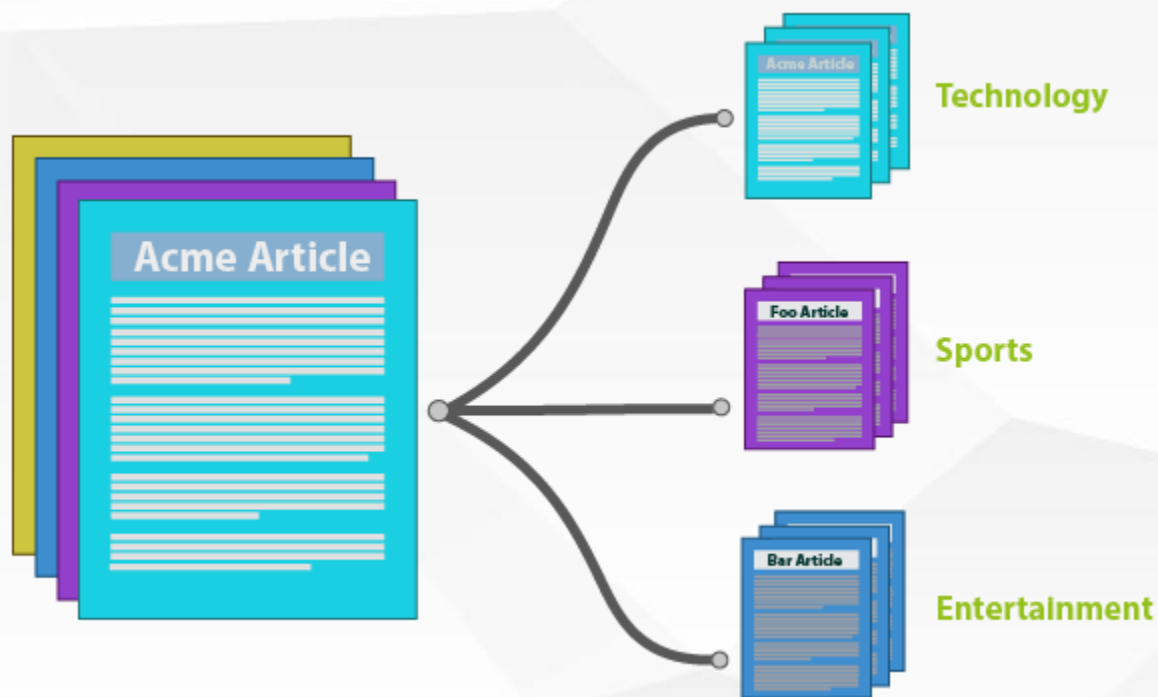
Course catalogue

1/ 机器学习概述

2/ 文本分类概述

3/ 实例

文本分类



什么是文本分类

文本分类(Text Classification或Text Categorization, TC), 或者称为自动文本分类(Automatic Text Categorization), 是指计算机将载有信息的一篇文本映射到预先给定的某一类别或某几类别主题的过程。

传统的分类方法

特征工程

特征工程在机器学习中往往是最耗时耗力的，但却极其的重要。抽象来讲，机器学习问题是把数据转换成信息再提炼到知识的过程，特征是“数据 $\rightarrow$ 信息”的过程，决定了结果的上限，而分类器是“信息 $\rightarrow$ 知识”的过程，则是去逼近这个上限。然而特征工程不同于分类器模型，不具备很强的通用性，往往需要结合对特征任务的理解。文本分类问题所在的自然语言领域自然也有其特有的特征处理逻辑，传统文本分类任务大部分工作也在此处。文本特征工程分为文本预处理、特征提取、文本表示三个部分，最终目的是把文本转换成计算机可理解的格式，并封装足够用于分类的信息，即很强的特征表达能力。

传统的分类方法

特征工程

1) 文本预处理

文本预处理过程是在文本中提取关键词表示文本的过程，中文文本处理中主要包括文本分词和去停用词两个阶段。之所以进行分词，是因为很多研究表明特征粒度为词粒度远好于字粒度，其实很好理解，因为大部分分类算法不考虑词序信息，基于字粒度显然损失了过多“n-gram”信息。具体到中文分词，不同于英文有天然的空格间隔，需要设计复杂的分词算法。传统算法主要有基于字符串匹配的正向/逆向/双向最大匹配；基于理解的句法和语义分析消歧；基于统计的互信息/CRF方法。

2) 特征提取

向量空间模型的文本表示方法的特征提取对应特征项的选择和特征权重计算两部分。特征选择的基本思路是根据某个评价指标独立的对原始特征项（词项）进行评分排序，从中选择得分最高的一些特征项，过滤掉其余的特征项。常用的评价有文档频率、互信息、信息增益、 χ^2 统计量等。特征权重主要是经典的TF-IDF方法及其扩展方法，主要思路是一个词的重要度与在类别内的词频成正比，与所有类别出现的次数成反比。

传统的分类方法

特征工程

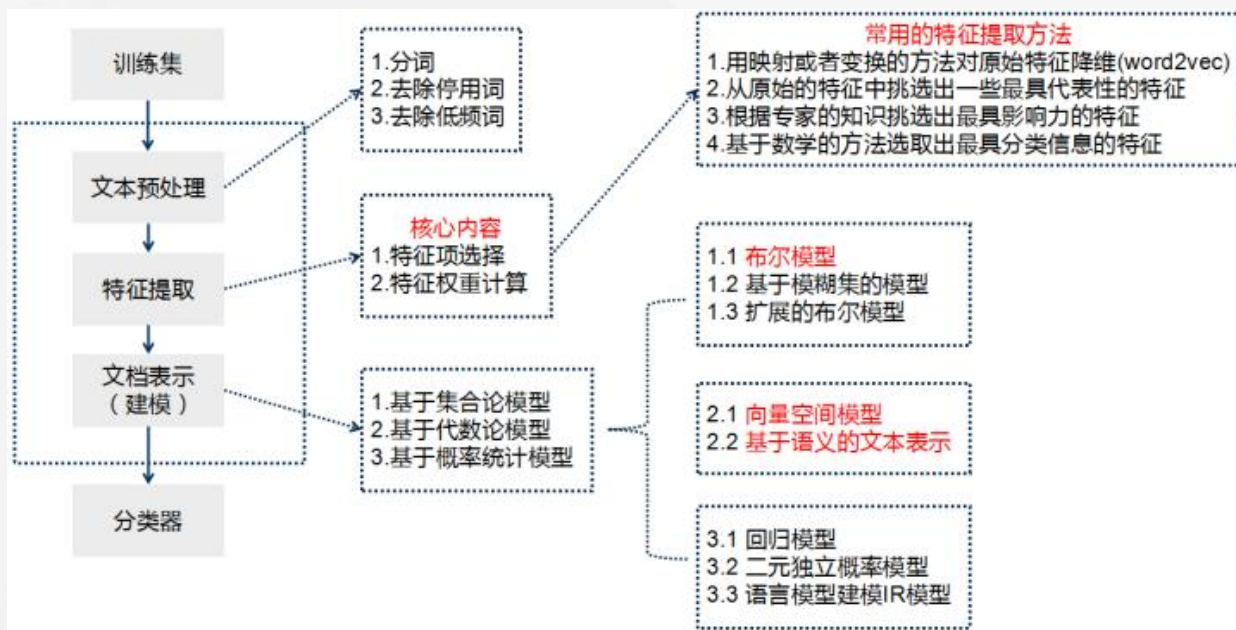
3) 文本表示

文本表示的目的是把文本预处理后的转换成计算机可理解的方式，是决定文本分类质量最重要的部分。

传统做法常用词袋模型（BOW, Bag Of Words）或向量空间模型（Vector Space Model），最大的不足是忽略文本上下文关系，每个词之间彼此独立，并且无法表征语义信息。

传统做法在文本表示方面除了向量空间模型，还有基于语义的文本表示方法，比如LDA主题模型、LSI/PLSI概率潜在语义索引等方法，一般认为这些方法得到的文本表示可以认为文档的深层表示，而word embedding文本分布式表示方法则是深度学习方法的重要基础。

传统的分类方法



分类器

分类器基本都是统计分类方法了，基本上大部分机器学习方法都在文本分类领域有所应用，比如朴素贝叶斯分类算法（Naïve Bayes）、KNN、SVM、最大熵和神经网络等等。

深度学习文本分类方法

深度学习文本分类方法

传统的文本分类做法的主要问题在于文本表示是高维度高稀疏的，特征表达能力很弱，而且神经网络很不擅长对此类数据的处理；此外需要人工进行特征工程，成本很高。而深度学习最初在之所以图像和语音取得巨大成功，一个很重要的原因是图像和语音原始数据是连续和稠密的，有局部相关性。应用深度学习解决大规模文本分类问题最重要的是解决文本表示，再利用CNN/RNN等网络结构自动获取特征表达能力，去掉繁杂的人工特征工程，端到端的解决问题。

练习时间

P78、 P82、 P88、 P91

课程目录

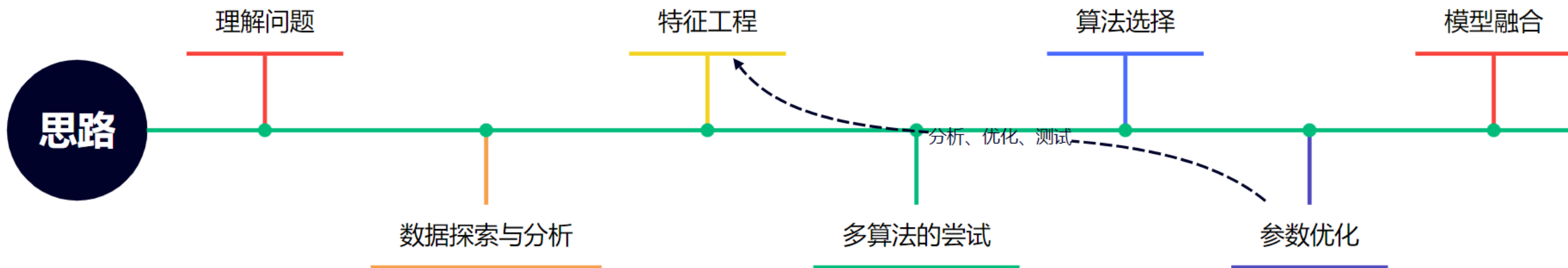
Course catalogue

1/ 机器学习概述

2/ 文本分类概述

3/ 实例

实例——一个回归问题



背景与目标
变量分析
数据预处理
特征处理
特征降维
异常值
归一化
...

回归模型
线性回归模型
K近邻回归模型
决策树回归模型
集成学习回归模型

理解问题

背景:

火力发电的基本原理是：燃料在燃烧时加热水生成蒸汽，蒸汽压力推动汽轮机旋转，然后汽轮机带动发电机旋转，产生电能。在这一系列的能量转化中，影响发电效率的核心是锅炉的燃烧效率，即燃料燃烧加热水产生高温高压蒸汽。锅炉的燃烧效率的影响因素很多，包括锅炉的可调参数，如燃烧给量，一二次风，引风，返料风，给水水量；以及锅炉的工况，比如锅炉床温、床压，炉膛温度、压力，过热器的温度等。

任务:

经脱敏后的锅炉传感器采集的数据（采集频率是分钟级别），根据锅炉的工况，预测产生的蒸汽量。

数据探索与分析

导库 & 数据

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5
6 from scipy import stats
7
8 import warnings
9 warnings.filterwarnings("ignore")
10
11 %matplotlib inline
```

```
1 train_data_file = "./data/zhengqi_train.txt"
2 test_data_file = "./data/zhengqi_test.txt"
3
4 train_data = pd.read_csv(train_data_file, sep='\t', encoding='utf-8')
5 test_data = pd.read_csv(test_data_file, sep='\t', encoding='utf-8')
```

训练集的基本信息

```
1 train_data.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2888 entries, 0 to 2887
Data columns (total 39 columns):
#   Column   Non-Null Count  Dtype  
---  -
0    V0        2888 non-null   float64
1    V1        2888 non-null   float64
2    V2        2888 non-null   float64
3    V3        2888 non-null   float64
4    V4        2888 non-null   float64
5    V5        2888 non-null   float64
6    V6        2888 non-null   float64
7    V7        2888 non-null   float64
8    V8        2888 non-null   float64
9    V9        2888 non-null   float64
10   V10       2888 non-null   float64
11   V11       2888 non-null   float64
12   V12       2888 non-null   float64
13   V13       2888 non-null   float64
14   V14       2888 non-null   float64
15   V15       2888 non-null   float64
16   V16       2888 non-null   float64
17   V17       2888 non-null   float64
18   V18       2888 non-null   float64
19   V19       2888 non-null   float64
20   V20       2888 non-null   float64
21   V21       2888 non-null   float64
22   V22       2888 non-null   float64
23   V23       2888 non-null   float64
24   V24       2888 non-null   float64
25   V25       2888 non-null   float64
26   V26       2888 non-null   float64
27   V27       2888 non-null   float64
28   V28       2888 non-null   float64
29   V29       2888 non-null   float64
30   V30       2888 non-null   float64
31   V31       2888 non-null   float64
32   V32       2888 non-null   float64
33   V33       2888 non-null   float64
34   V34       2888 non-null   float64
35   V35       2888 non-null   float64
36   V36       2888 non-null   float64
37   V37       2888 non-null   float64
38   target    2888 non-null   float64
dtypes: float64(39)
memory usage: 880.1 KB
```

V0~V37是特征变量， target是目标变量

训练集的统计信息

1 #查看数据统计信息
2 train_data.describe()

	V0	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V29
count	2888.000000	2888.000000	2888.000000	2888.000000	2888.000000	2888.000000	2888.000000	2888.000000	2888.000000	2888.000000	...	2888.000000
mean	0.123048	0.056068	0.289720	-0.067790	0.012921	-0.558565	0.182892	0.116155	0.177856	-0.169452	...	0.097648
std	0.928031	0.941515	0.911236	0.970298	0.888377	0.517957	0.918054	0.955116	0.895444	0.953813	...	1.061200
min	-4.335000	-5.122000	-3.420000	-3.956000	-4.742000	-2.182000	-4.576000	-5.048000	-4.692000	-12.891000	...	-2.912000
25%	-0.297000	-0.226250	-0.313000	-0.652250	-0.385000	-0.853000	-0.310000	-0.295000	-0.159000	-0.390000	...	-0.664000
50%	0.359000	0.272500	0.386000	-0.044500	0.110000	-0.466000	0.388000	0.344000	0.362000	0.042000	...	-0.023000
75%	0.726000	0.599000	0.918250	0.624000	0.550250	-0.154000	0.831250	0.782250	0.726000	0.042000	...	0.745250
max	2.121000	1.918000	2.828000	2.457000	2.689000	0.489000	1.895000	1.918000	2.245000	1.335000	...	4.580000

8 rows × 39 columns

训练集的字段信息

1 #查看数据字段信息
2 train_data.head()

	V0	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V29	V30	V31	V32	V33	V34	V35	V36	V37	target
0	0.566	0.016	-0.143	0.407	0.452	-0.901	-1.812	-2.360	-0.436	-2.114	...	0.136	0.109	-0.615	0.327	-4.627	-4.789	-5.101	-2.608	-3.508	0.175
1	0.968	0.437	0.066	0.566	0.194	-0.893	-1.566	-2.360	0.332	-2.114	...	-0.128	0.124	0.032	0.600	-0.843	0.160	0.364	-0.335	-0.730	0.676
2	1.013	0.568	0.235	0.370	0.112	-0.797	-1.367	-2.360	0.396	-2.114	...	-0.009	0.361	0.277	-0.116	-0.843	0.160	0.364	0.765	-0.589	0.633
3	0.733	0.368	0.283	0.165	0.599	-0.679	-1.200	-2.086	0.403	-2.114	...	0.015	0.417	0.279	0.603	-0.843	-0.065	0.364	0.333	-0.112	0.206
4	0.684	0.638	0.260	0.209	0.337	-0.454	-1.073	-2.086	0.314	-2.114	...	0.183	1.078	0.328	0.418	-0.843	-0.215	0.364	-0.280	-0.028	0.384

5 rows × 39 columns

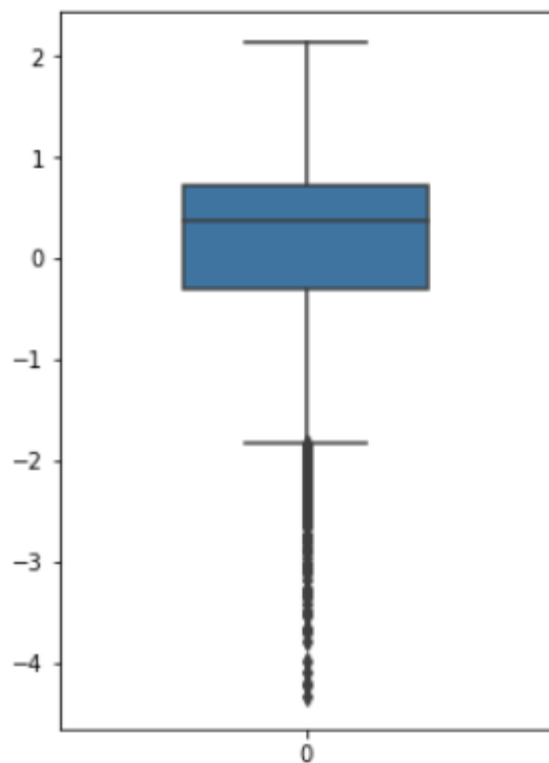
同时，查看下测试集

可视化数据分布

例如，箱型图

```
1 #可视化
2 fig = plt.figure(figsize=(4, 6))
3 sns.boxplot(train_data['V0'], orient='v', width=0.5)
```

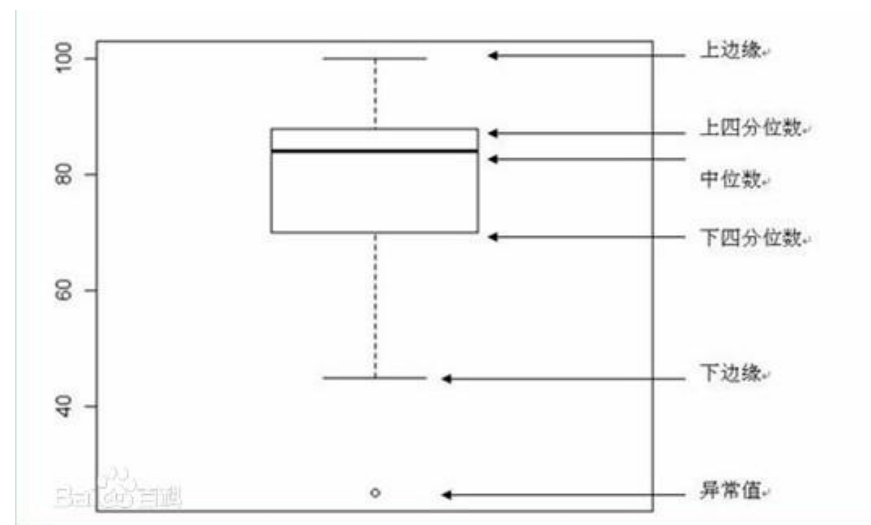
<AxesSubplot: >



同时，画出其他特征

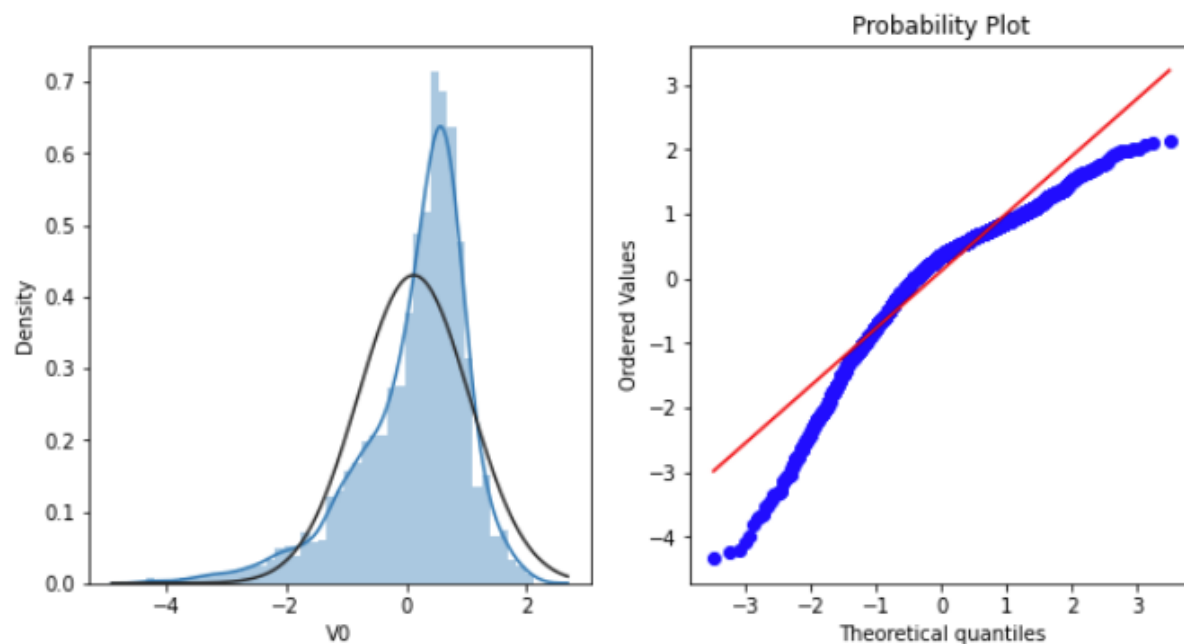
箱形图（英文：**Box plot**），又称为盒须图、盒式图、盒状图或箱线图，是一种用作显示一组数据分散情况资料的统计图。因形状如箱子而得名。在各种领域也经常被使用，常见于品质管理。不过作法相对较繁琐。

箱形图于**1977**年由美国著名统计学家约翰·图基（**John Tukey**）发明。它能显示出一组数据的最大值、最小值、中位数、及上下四分位数。



QQ图

```
1 plt.figure(figsize=(10,5))
2
3 ax=plt.subplot(1,2,1)
4 sns.distplot(train_data['V0'],fit=stats.norm)
5 ax=plt.subplot(1,2,2)
6 res = stats.probplot(train_data['V0'], plot=plt)
```

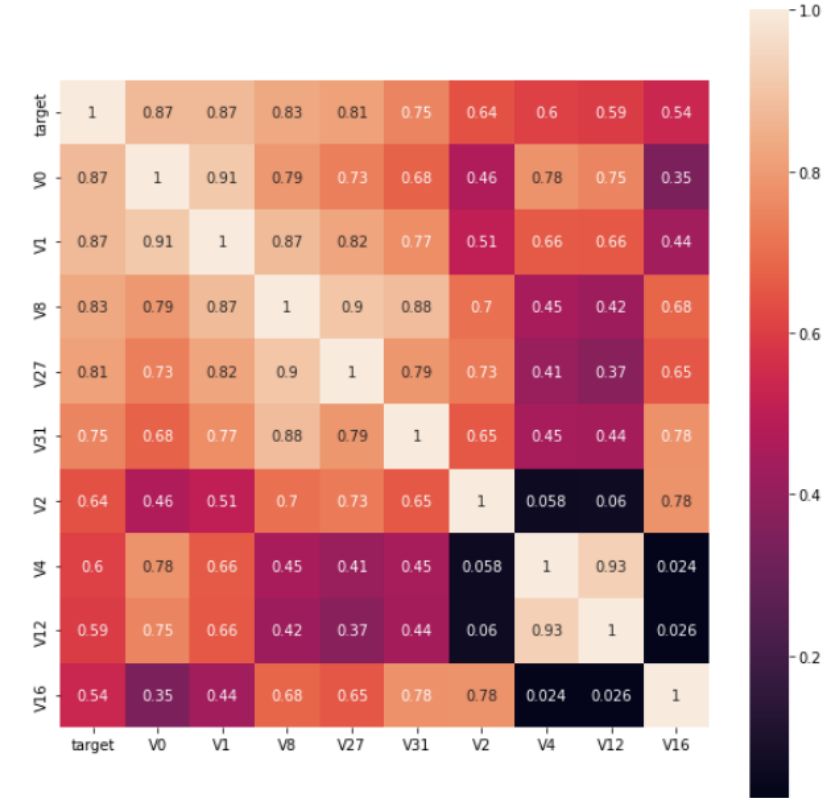


V0不是正态分布
检查其他变量

[QQ图法检验正态分布 - 百度文库 \(baidu.com\)](http://baidu.com)

计算相关系数 筛选特征变量

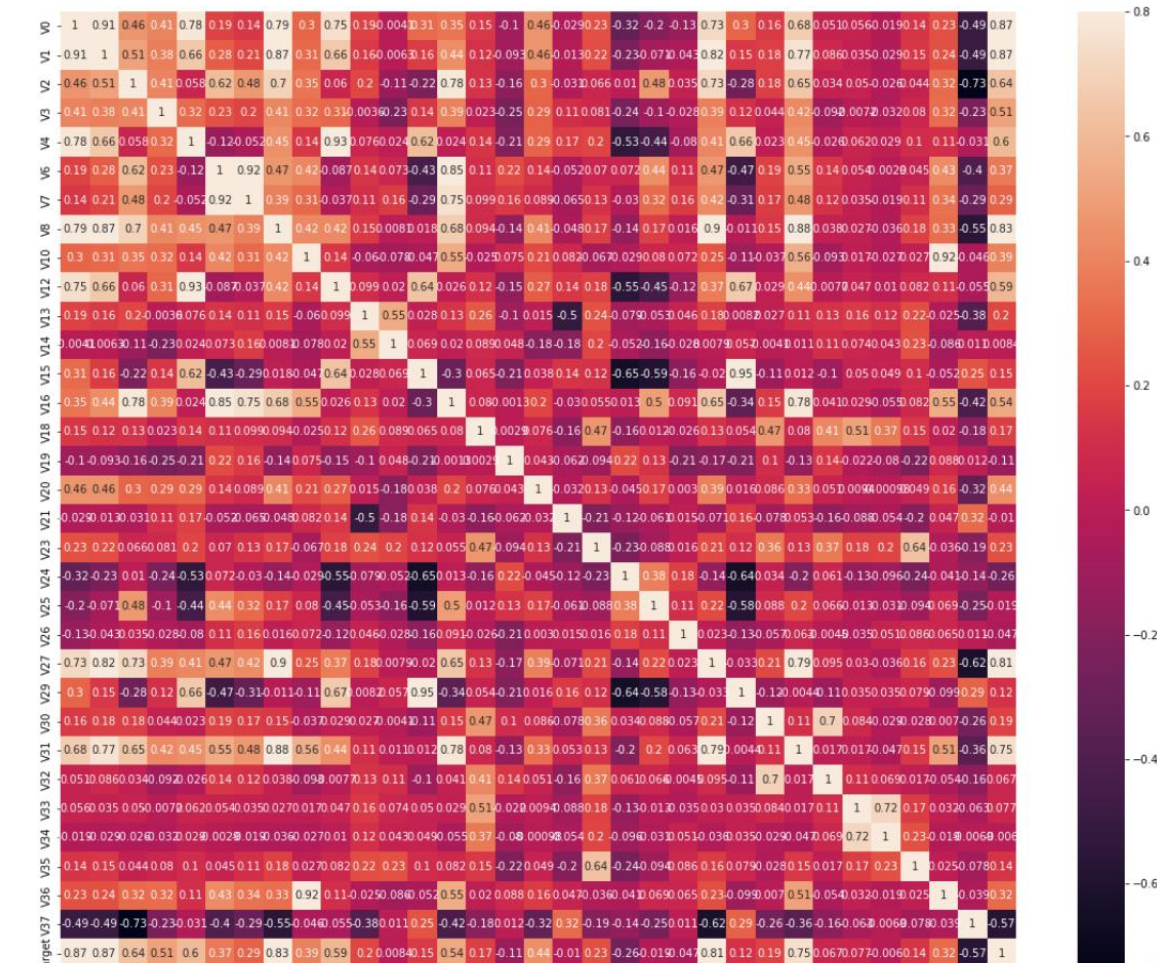
```
1 #寻找K个最相关的特征信息
2 k = 10 # number of variables for heatmap
3 cols = train_corr.nlargest(k, 'target')['target'].index
4
5 cm = np.corrcoef(train_data[cols].values.T)
6 hm = plt.subplots(figsize=(10, 10)) #调整画布大小
7 #hm = sns.heatmap(cm, cbar=True, annot=True, square=True)
8 #g = sns.heatmap(train_data[cols].corr(), annot=True, square=True, cmap="RdYlGn")
9 hm = sns.heatmap(train_data[cols].corr(), annot=True, square=True)
10
11 plt.show()
```



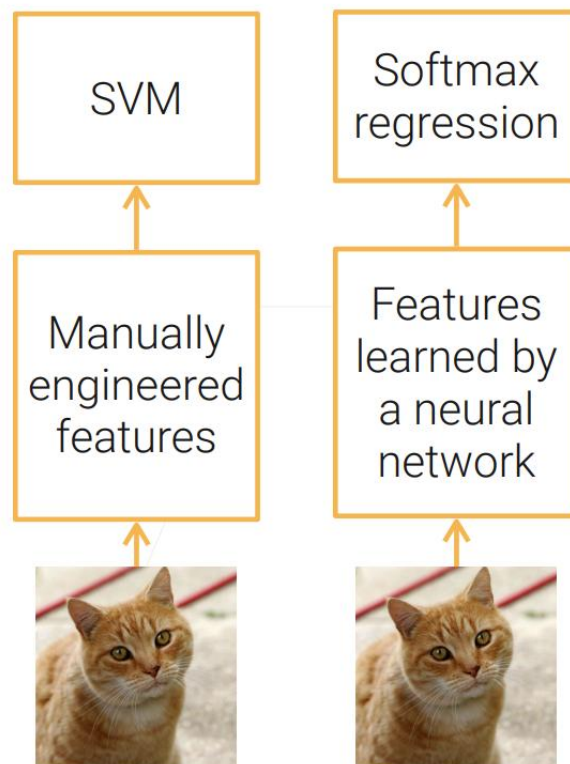
```
1 data_train1 = train_data.drop(['V5', 'V9', 'V11', 'V17', 'V22', 'V28'], axis=1)
2 train_corr = data_train1.corr()
3 train_corr
```

	V0	V1	V2	V3	V4	V6	V7	V8	V10	V12	...	V29	V30	V31	V
V0	1.000000	0.908607	0.463643	0.409576	0.781212	0.189267	0.141294	0.794013	0.298443	0.751830	...	0.302145	0.156968	0.675003	0.0501
V1	0.908607	1.000000	0.506514	0.383924	0.657790	0.276805	0.205023	0.874650	0.310120	0.656186	...	0.147096	0.175997	0.769745	0.0851
V2	0.463643	0.506514	1.000000	0.410148	0.057697	0.615938	0.477114	0.703431	0.346006	0.059941	...	-0.275764	0.175943	0.653764	0.0331
V3	0.409576	0.383924	0.410148	1.000000	0.315046	0.233896	0.197836	0.411946	0.321262	0.306397	...	0.117610	0.043966	0.421954	-0.0921
V4	0.781212	0.657790	0.057697	0.315046	1.000000	-0.117529	-0.052370	0.449542	0.141129	0.927685	...	0.659093	0.022807	0.447016	-0.0261
V6	0.189267	0.276805	0.615938	0.233896	-0.117529	1.000000	0.917502	0.468233	0.415660	-0.087312	...	-0.467980	0.188907	0.546535	0.1441
V7	0.141294	0.205023	0.477114	0.197836	-0.052370	0.917502	1.000000	0.389987	0.310982	-0.036791	...	-0.311363	0.170113	0.475254	0.1221
V8	0.794013	0.874650	0.703431	0.411946	0.449542	0.468233	0.389987	1.000000	0.419703	0.420557	...	-0.011091	0.150258	0.878072	0.0381
V10	0.298443	0.310120	0.346006	0.321262	0.141129	0.415660	0.310982	0.419703	1.000000	0.140462	...	-0.105042	-0.036705	0.560213	-0.0931

```
1 # 画出相关性热力图
2 ax = plt.subplots(figsize=(20, 16)) #调整画布大小
3
4 ax = sns.heatmap(train_corr, vmax=.8, square=True, annot=True) #画热力图  annot=True 显示系数
```



特征工程



Before deep learning (DL), feature engineering (FE) was critical to using ML models.

特征处理方法

标准化是依照特征矩阵的列处理数据，即通过求标准分数的方法，将特征转换为标准正态分布^Q，并和整体样本分布相关。每一个样本点都能对标准化产生影响。

标准化需要计算特征的均值和标准差，公式如下：

$$x' = \frac{x - \bar{X}}{S}$$

区间缩放法的思路有很多种，常见的一种是利用两个最值（最大值和最小值）进行缩放，公式如下：

$$x' = \frac{x - Min}{Max - Min}$$

归一化是将样本的特征值转换到同一量纲下，把数据映射到[0,1]或者[a,b]区间内，由于其仅由变量的极值决定，因此区间缩放法是一种归一化的方法。

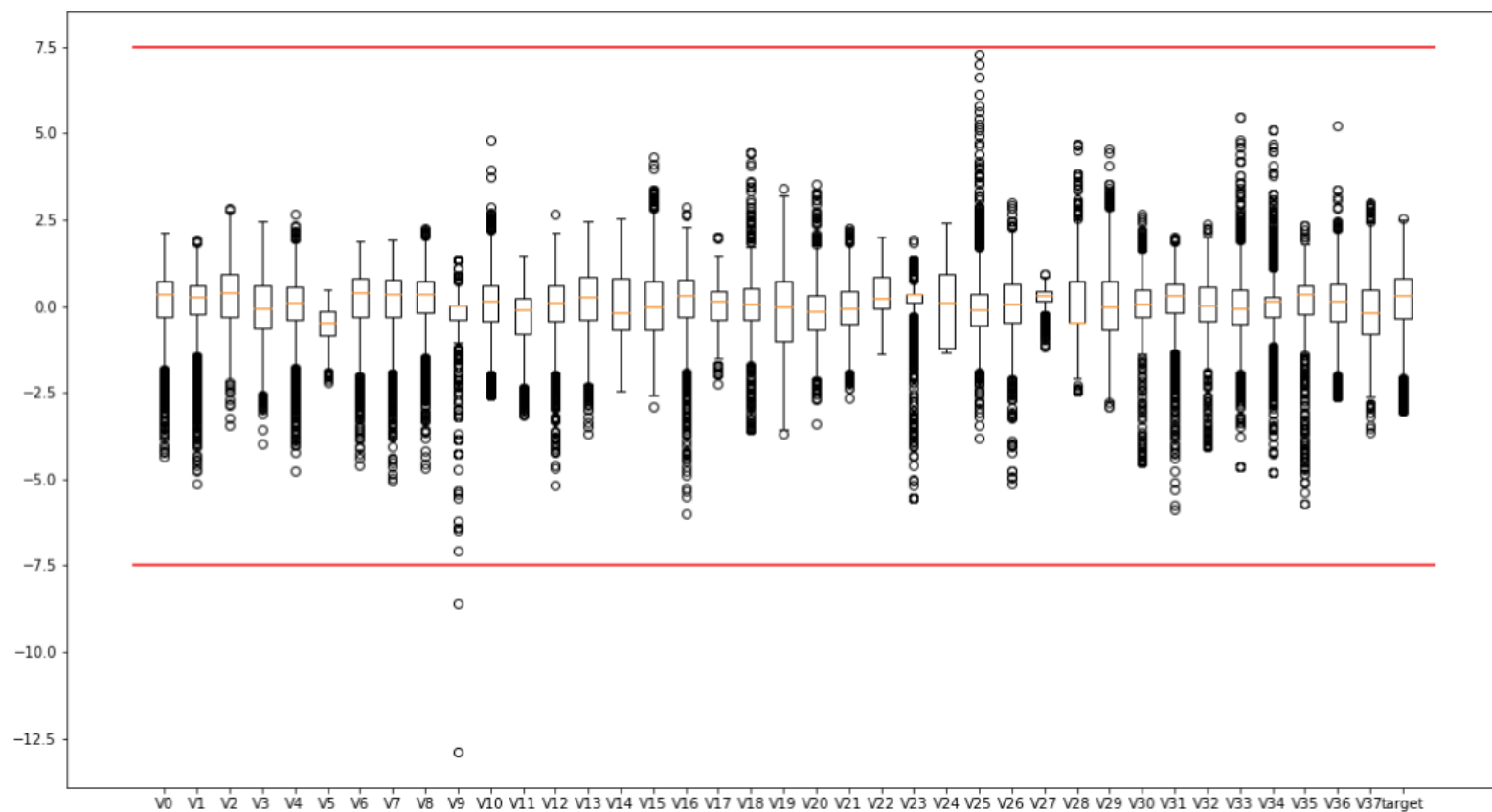
归一区间会改变数据的原始距离、分布和信息，但标准化一般不会。

规则为L2的归一化公式如下：

$$x' = \frac{x}{\sqrt{\sum_j^m x[j]^2}}$$

异常值分析与删除

```
1 #异常值分析
2 plt.figure(figsize=(18,10))
3 plt.boxplot(x=train_data.values, labels=train_data.columns)
4 plt.hlines([-7.5, 7.5], 0, 40, colors='r')
5 plt.show()
```



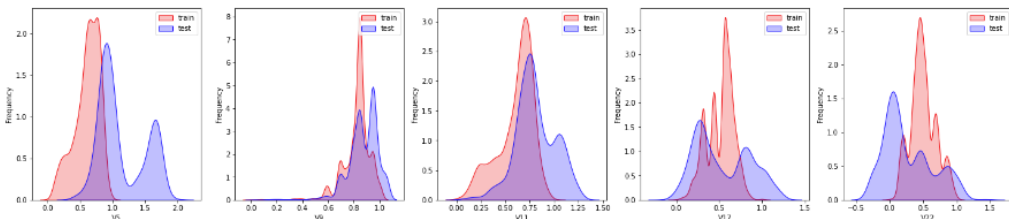
```
1 #删除异常值
2 train_data=train_data[train_data['V9']>-7.5]
```

对数据进行归一化

```
1 #最大最小值归一化
2 from sklearn import preprocessing
3 features_columns = [col for col in train_data.columns if col not in ['target']]
4 min_max_scaler = preprocessing.MinMaxScaler()
5 min_max_scaler = min_max_scaler.fit(train_data[features_columns])
6 train_data_scaler = min_max_scaler.transform(train_data[features_columns])
7 test_data_scaler = min_max_scaler.transform(test_data[features_columns])
8
9 train_data_scaler = pd.DataFrame(train_data_scaler)
10 train_data_scaler.columns = features_columns
11
12 test_data_scaler = pd.DataFrame(test_data_scaler)
13 test_data_scaler.columns = features_columns
14
15 train_data_scaler['target'] = train_data['target']
```

查看数据分布

```
1 drop_col = 6
2 drop_row = 1
3
4 plt.figure(figsize=(5*drop_col, 5*drop_row))
5
6 for i, col in enumerate(['V5', 'V9', 'V11', 'V17', 'V22', 'V28']):
7     ax = plt.subplot(drop_row, drop_col, i+1)
8     ax = sns.kdeplot(train_data_scaler[col], color='Red', shade=True)
9     ax = sns.kdeplot(test_data_scaler[col], color='Blue', shade=True)
10    ax.set_xlabel(col)
11    ax.set_ylabel('Frequency')
12    ax = ax.legend(['train', 'test'])
13 plt.show()
```



```
1 #特征相关性
2 plt.figure(figsize=(20, 16))
3 column = train_data_scaler.columns.tolist()
```

计算特征相关性，使用PCA去除多种共线性

```
#采用 pca 保留16维特征的数据
new_train_pca_16 = new_train_pca_16.fillna(0)
train = new_train_pca_16[new_test_pca_16.columns]
target = new_train_pca_16['target']
```

Try多算法

导包

```
1 #导包
2 from sklearn.linear_model import LinearRegression #线性回归
3 from sklearn.neighbors import KNeighborsRegressor #K近邻回归
4 from sklearn.tree import DecisionTreeRegressor #决策树回归
5 from sklearn.ensemble import RandomForestRegressor #随机森林回归
6 from sklearn.svm import SVR #支持向量回归
7 import lightgbm as lgb #lightGbm模型
8
9 from sklearn.model_selection import train_test_split #切分数据
10 from sklearn.metrics import mean_squared_error #评价指标
11
12 from sklearn.model_selection import learning_curve
13 from sklearn.model_selection import ShuffleSplit
```

切分数据集

```
# 切分数据 训练数据80% 验证数据20%
train_data, test_data, train_target, test_target=train_test_split(train, target, test_size=0.2, random_state=0)
```

多元线性回归

```
1 clf = LinearRegression()
2 clf.fit(train_data, train_target)
3 score = mean_squared_error(test_target, clf.predict(test_data))
4 print("LinearRegression: ", score)
```

LinearRegression: 0.2642337917628173

K近邻回归

```
1 clf = KNeighborsRegressor(n_neighbors=8) # 最近三个
2 clf.fit(train_data, train_target)
3 score = mean_squared_error(test_target, clf.predict(test_data))
4 print("KNeighborsRegressor: ", score)
```

KNeighborsRegressor: 0.2635369464478806

随机森林

```
1 clf = RandomForestRegressor(n_estimators=200) # 200棵树模型
2 clf.fit(train_data, train_target)
3 score = mean_squared_error(test_target, clf.predict(test_data))
4 print("RandomForestRegressor: ", score)
```

RandomForestRegressor: 0.24944834690782874

模型验证与调参

欠拟合、过拟合与正常拟合

```
1 clf = SGDRegressor(max_iter=500, tol=1e-2)
2 clf.fit(train_data, train_target)
3 score_train = mean_squared_error(train_target, clf.predict(train_data))
4 score_test = mean_squared_error(test_target, clf.predict(test_data))
5 print("SGDRegressor train MSE: ", score_train)
6 print("SGDRegressor test MSE: ", score_test)
```

SGDRegressor train MSE: 0.15131539419626705
SGDRegressor test MSE: 0.15582142312430822

```
1 from sklearn.preprocessing import PolynomialFeatures
2 poly = PolynomialFeatures(5)
3 train_data_poly = poly.fit_transform(train_data)
4 test_data_poly = poly.transform(test_data)
5 clf = SGDRegressor(max_iter=1000, tol=1e-3)
6 clf.fit(train_data_poly, train_target)
7 score_train = mean_squared_error(train_target, clf.predict(train_data_poly))
8 score_test = mean_squared_error(test_target, clf.predict(test_data_poly))
9 print("SGDRegressor train MSE: ", score_train)
10 print("SGDRegressor test MSE: ", score_test)
```

SGDRegressor train MSE: 0.13236433435249664
SGDRegressor test MSE: 0.14493591005137052

```
1 from sklearn.preprocessing import PolynomialFeatures
2 poly = PolynomialFeatures(3)
3 train_data_poly = poly.fit_transform(train_data)
4 test_data_poly = poly.transform(test_data)
5 clf = SGDRegressor(max_iter=1000, tol=1e-3)
6 clf.fit(train_data_poly, train_target)
7 score_train = mean_squared_error(train_target, clf.predict(train_data_poly))
8 score_test = mean_squared_error(test_target, clf.predict(test_data_poly))
9 print("SGDRegressor train MSE: ", score_train)
10 print("SGDRegressor test MSE: ", score_test)
```

SGDRegressor train MSE: 0.13394644851833623
SGDRegressor test MSE: 0.14239861550563235

		Training error	
		Low	High
Generalization error	Low	Good	Bug?
	High	Overfitting	Underfitting

正则化

```
1 #模型正则化
2 poly = PolynomialFeatures(3)
3 train_data_poly = poly.fit_transform(train_data)
4 test_data_poly = poly.transform(test_data)
5 clf = SGDRegressor(max_iter=1000, tol=1e-3, penalty='L2', alpha=0.0001)
6 clf.fit(train_data_poly, train_target)
7 score_train = mean_squared_error(train_target, clf.predict(train_data_poly))
8 score_test = mean_squared_error(test_target, clf.predict(test_data_poly))
9 print("SGDRegressor train MSE: ", score_train)
10 print("SGDRegressor test MSE: ", score_test)
```

SGDRegressor train MSE: 0.13421912889219112
SGDRegressor test MSE: 0.14251903162694787

```
1 poly = PolynomialFeatures(3)
2 train_data_poly = poly.fit_transform(train_data)
3 test_data_poly = poly.transform(test_data)
4 clf = SGDRegressor(max_iter=1000, tol=1e-3, penalty='L1', alpha=0.00001)
5 clf.fit(train_data_poly, train_target)
6 score_train = mean_squared_error(train_target, clf.predict(train_data_poly))
7 score_test = mean_squared_error(test_target, clf.predict(test_data_poly))
8 print("SGDRegressor train MSE: ", score_train)
9 print("SGDRegressor test MSE: ", score_test)
```

SGDRegressor train MSE: 0.13419495528305822
SGDRegressor test MSE: 0.14268759279867613

交叉验证

```
1 # 简单交叉验证
2 from sklearn.model_selection import train_test_split # 切分数据
3 # 切分数据 训练数据80% 验证数据20%
4 train_data, test_data, train_target, test_target=train_test_split(train, target, test_size=0.2, random_state=0)
5
6 clf = SGDRegressor(max_iter=1000, tol=1e-3)
7 clf.fit(train_data, train_target)
8 score_train = mean_squared_error(train_target, clf.predict(train_data))
9 score_test = mean_squared_error(test_target, clf.predict(test_data))
10 print("SGDRegressor train MSE: ", score_train)
11 print("SGDRegressor test MSE: ", score_test)
```

SGDRegressor train MSE: 0.14150093021366658
SGDRegressor test MSE: 0.14688489683160996

```
1 # 5折交叉验证
2 from sklearn.model_selection import KFold
3
4 kf = KFold(n_splits=5)
5 for k, (train_index, test_index) in enumerate(kf.split(train)):
6     train_data, test_data, train_target, test_target = train.values[train_index], train.values[test_index], target[train_index], target[test_index]
7     clf = SGDRegressor(max_iter=1000, tol=1e-3)
8     clf.fit(train_data, train_target)
9     score_train = mean_squared_error(train_target, clf.predict(train_data))
10    score_test = mean_squared_error(test_target, clf.predict(test_data))
11    print(k, " 折", "SGDRegressor train MSE: ", score_train)
12    print(k, " 折", "SGDRegressor test MSE: ", score_test, '\n')
```

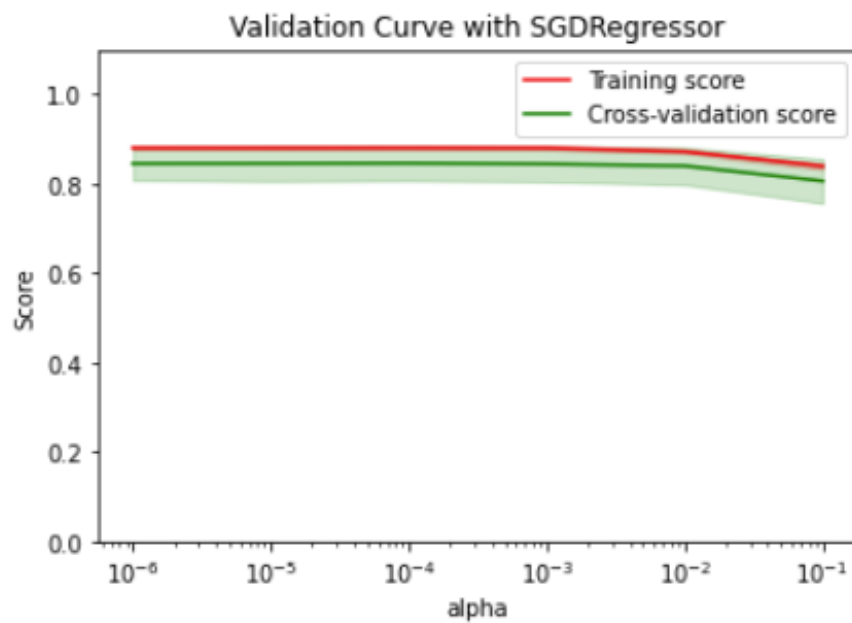
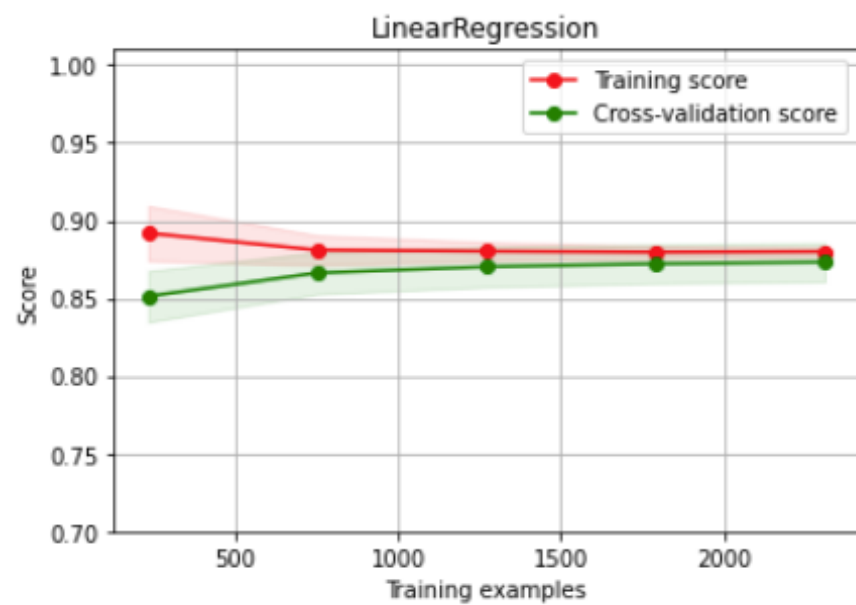
网格搜索

```
1 # 穷举网格搜索
2 from sklearn.model_selection import GridSearchCV
3 from sklearn.ensemble import RandomForestRegressor
4 from sklearn.model_selection import train_test_split # 切分数据
5 # 切分数据 训练数据80% 验证数据20%
6 train_data, test_data, train_target, test_target=train_test_split(train, target, test_size=0.2, random_state=0)
7
8 randomForestRegressor = RandomForestRegressor()
9 parameters = {
10     'n_estimators':[50, 100, 200],
11     'max_depth':[1, 2, 3]
12 }
13
14
15 clf = GridSearchCV(randomForestRegressor, parameters, cv=5)
16 clf.fit(train_data, train_target)
17
18 score_test = mean_squared_error(test_target, clf.predict(test_data))
19
20 print("RandomForestRegressor GridSearchCV test MSE: ", score_test)
21 sorted(clf.cv_results_.keys())
```

RandomForestRegressor GridSearchCV test MSE: 0.25703588289789986

```
['mean_fit_time',
 'mean_score_time',
 'mean_test_score',
 'param_max_depth',
 'param_n_estimators',
 'params',
 'rank_test_score',
 'split0_test_score',
 'split1_test_score',
 'split2_test_score',
 'split3_test_score',
 'split4_test_score',
 'std_fit_time',
 'std_score_time',
 'std_test_score']
```

学习曲线



结束

谢谢